INOVIQ — Comprehensive College Viva Preparation Guide

Tactile Digital Visiting Card Catalog & Studio • Full Architecture, Codebase & Contribution Breakdown

Student / Author: Riddhi Gandhi (`riddhii24`) **Email:** riddhigandhi72@gmail.com

Total Commits: 25 commits **Repository:** Inoviq (v2.0.0) **Date:** October 2026

1. Project Overview, Tech Stack & Folder Structure

1.1 What the Project Does

INOVIQ is an interactive web platform that modernizes professional networking by replacing physical paper business cards with tactile digital cards. Users can design digital cards in an interactive 3D studio, share credentials instantly via smartphone-scannable QR codes (vCard format), scan physical visiting cards using their camera via `Html5Qrcode`, and store and search their collection in a categorized digital vault. It features a seamless **Guest vs. Authenticated Member** workflow: unauthenticated guests can explore the templates and play with card customization, while sensitive features (cloud storage, personal card catalog, camera scanner) are protected by an authentication gate modal.

1.2 Tech Stack Table

Layer	Technology	Key Purpose & Architecture
Frontend Hybrid	**AngularJS (1.8.3)** + **React 18**	• **AngularJS**: Single-page navigation, controllers (`DashboardController`, `SavedCardsController`, `ScanController`, `AuthController`), and API services. • **React 18**: Embedded in `create-card.html` for the high-performance 3D Card Studio.
Styling & 3D Engine	**Vanilla CSS3** (Variables & 3D Transforms)	Tactile design tokens (Plaster, Soot, Mist, Eucalyptus), CSS 3D perspective transforms (`perspective: 1200px`, `rotateX`, `rotateY`), dynamic specular glare, and light/dark theme toggles.
Client Libraries	`qrcodejs`, `html5-qrcode`, `html2canvas`	• `qrcodejs`: Generates scannable vCard/MECARD QR codes client-side. • `html5-qrcode`: Live camera feed decoding for QR and barcodes. • `html2canvas`: 3x high-DPI rasterization for PNG card downloads.
Backend Server	**Node.js** & **Express.js (v4.22)**	RESTful API endpoints (`/api/auth`, `/api/cards`), static asset serving, and no-cache live development middleware.
Database & ODM	**MongoDB** & **Mongoose (v9.10)**	Atlas cloud connection with an automatic local fallback (`mongodb://127.0.0.1:27017/inoviq`) to guarantee zero offline downtime.
Security	**JWT**, **bcryptjs**, **cookie-parser**	Salted password hashing (cost factor 12) and stateless Bearer token authorization via custom Express middleware.

1.3 Directory Structure

```
Inoviq/
├── Client/                          # Client-side architecture
│   ├── Pages/                       # View templates
│   │   ├── dashboard.html           # Landing page, hero 3D card stage, animated search pill
│   │   ├── create-card.html         # React 18 3D Card Studio (Edit Mode vs 3D Preview)
│   │   ├── saved-cards.html         # Saved card catalog, filter tabs & 3D inspect modal
│   │   ├── scan.html                # Camera scanner (Html5Qrcode) & vCard importer
│   │   ├── templates.html           # Pre-designed card templates showcase
│   │   ├── profile.html             # User profile edit & personal statistics
│   │   ├── login.html               # Login page
│   │   └── signup.html              # Signup page with password validation
│   ├── Style/                       # Modular CSS stylesheets
│   │   ├── style.css                # Master tactile design tokens, navbar, themes
│   │   ├── studio.css               # 3D Card Studio layout, tilt physics, glare
│   │   ├── dashboard.css            # Catalog and grid styling
│   │   └── auth-modal.css           # Guest auth modal & backdrop styles
│   ├── services/                    # AngularJS Services
│   │   ├── auth.service.js          # Guest/User state, localStorage session, mock JWT
│   │   └── auth-modal.service.js    # Modal event bus ($rootScope.$broadcast)
│   ├── components/                  # Directives
│   │   └── auth-gate.component.js   # <auth-gate> modal directive
│   ├── routes.config.js             # ui-router configuration & route protection interceptor
│   ├── app.js                       # Main AngularJS app, controllers & CardService
│   └── index.html                   # Root redirector to Pages/dashboard.html
├── Server/                          # Node.js Express Backend
│   ├── config/db.js                 # Mongoose connection with Atlas/Local fallback
│   ├── models/User.js               # User Schema + pre-save bcrypt hash + comparePassword
│   ├── models/Card.js               # Card Schema + pre-save initials generator
│   ├── routes/auth.js               # /api/auth (signup, login, me, profile)
│   ├── routes/cards.js              # /api/cards (CRUD operations with authMiddleware)
│   ├── middleware/auth.js           # JWT Bearer token validator middleware
│   └── server.js                    # Main Express server entry point (port 3000)
├── package.json                     # Scripts and project dependencies
└── README.md                        # Project documentation
```

2. How Main Features Work (Files & Functions Behind Them)

2.1 Tactile 3D Card Studio (Edit vs. 3D Preview)

- **Files:** `Client/Pages/create-card.html`, `Client/Style/studio.css`
- **How it works:**
  - Built with an embedded React 18 component mounted onto `#react-card-studio-root`.
  - **Dual Modes:** In *Edit Mode*, card transformation is frozen (`transform: none`) so typing, dragging logos, and styling can be done millimeter-precise without cursor interference. In *3D Preview Mode*, mouse movement over the stage dynamically tilts the card in 3D perspective: `rotateY = ((x - centerX) / width) * 25deg`, `rotateX = -((y - centerY) / height) * 25deg`, while shifting a radial gradient specular reflection layer across the card face.
  - **PNG Export:** Calls `html2canvas` to rasterize the card DOM at 3x resolution into a high-DPI downloadable image.
  - **Live QR:** Instantiates `new QRCode(container, { text: vCardString })` to generate live SVG/Canvas QR codes client-side.

2.2 Guest Authentication & Route Gatekeeping

- **Files:** `Client/services/auth.service.js`, `Client/services/auth-modal.service.js`, `Client/components/auth-gate.component.js`, `Client/routes.config.js`
- **How it works:**
  - `AuthService.isLoggedIn()` inspects `localStorage.getItem('inoviq_logged_in')` and checks token expiration. Default state is unauthenticated guest.
  - `routes.config.js` defines state routes and attaches `$transitions.onStart`. If a destination state has `data: { requiresAuth: true }` and the user is a guest:
    1. It saves the blocked destination via `AuthService.setRedirectTarget(toState.name)`.
    2. It invokes `AuthModalService.open(...)` which broadcasts `'authGate:open'` across `$rootScope`.
    3. The `<auth-gate>` directive catches the event and displays the modal with backdrop and Escape key listener.
    4. Navigation is cancelled via `return false`. Once the user logs in or signs up, they are automatically returned to their original destination.

2.3 Digital Card Catalog & 3D Inspect Modal

- **Files:** `Client/Pages/saved-cards.html`, `Client/app.js` (`SavedCardsController`, lines 1013–1327)
- **How it works:**
  - Loads cards through `CardService.list()` which makes an authenticated `GET /api/cards` request. If offline or guest, it loads built-in showcase cards (`defaultShowcase`) so the UI remains demonstrable.
  - `catalog.filteredCards()` performs real-time client-side filtering matching search terms across `fullName`, `company`, `jobTitle`, and `email`, combined with category tabs (All, Business, Science, Creative).
  - Clicking a card invokes `catalog.openInspect(card)` which triggers an interactive 3D inspection modal. Calling `catalog.toggleModalCardFlip()` applies a CSS `transform: rotateY(180deg)` to reveal the back of the card and its live scannable QR code.

2.4 Live Camera QR Scanner & vCard Importer

- **Files:** `Client/Pages/scan.html`, `Client/app.js` (`ScanController`, lines 1328–1724)
- **How it works:**
  - Instantiates `Html5Qrcode("qr-video-reader")` requesting the rear environment camera (`facingMode: 'environment'`).
  - On QR detection, `scan.onCodeDecoded(decodedText)` parses `vCard` (e.g. `FN:`, `ORG:`, `TITLE:`, `TEL:`, `EMAIL:`) or `MECARD` payloads using regex into card object fields.
  - Clicking "Save to Catalog" issues `CardService.create(scan.card)` via `POST /api/cards` to store the card in MongoDB.

2.5 Streamlined Topbar Navigation & Expandable Search Pill

- **Files:** `Client/Pages/dashboard.html`, `Client/Style/style.css`
- **How it works:**
  - Provides a clean, uncluttered brand header housing the Inoviq logo, primary links (Templates, Scan, Saved Cards), theme toggle, and auth status.
  - **Animated Search Pill:** Starts as a compact 44px circular button with a search magnifying icon. Clicking it fires a `dash-ripple` circular CSS expansion wave and smoothly expands into a 320px input field using cubic-bezier easing.
  - Keyboard accessibility: Pressing `Enter` expands the search, and pressing `Escape` or clicking outside collapses it back while syncing with AngularJS `searchTerm` models.

3. Database Schema & REST API Endpoints

3.1 Database Schemas (MongoDB / Mongoose)

1. User Schema (`Server/models/User.js`):

Field	Type	Rules & Defaults	Description
`firstName`	String	Required, trim, max 50	User's first name
`lastName`	String	Required, trim, max 50	User's last name
`email`	String	Required, unique, lowercase, regex validated	Account login email
`username`	String	Unique, sparse, min 3, max 30	Optional unique user handle
`phone`	String	Trim, default `""`	Contact phone number
`password`	String	Required, min 8 characters	Bcrypt salted hash (cost factor 12)

Hooks: `userSchema.pre('save')` automatically hashes passwords. `userSchema.methods.comparePassword` verifies passwords via `bcrypt.compare`. `userSchema.set('toJSON')` scrubs `password` and `__v` from API outputs.

2. Card Schema (`Server/models/Card.js`):

Field	Type	Rules & Defaults	Description
`userId`	ObjectId	Ref: `'User'`, Required, Indexed	Foreign key pointing to the card owner
`fullName`	String	Required, max 100	Cardholder name
`jobTitle`	String	Default `""`	Professional position/title
`company`	String	Default `"Independent"`, max 100	Organization name
`email`, `phone`, `website`	String	Default `""`	Contact credentials
`templateLabel`	String	Default `"Ledger"`	Visual style category label
`color`, `accentColor`, `bgGradient`	String	Hex & CSS gradient strings	Tactile styling attributes
`initials`	String	Max 3 chars	Auto-computed monogram badge letters

Hooks: `cardSchema.pre('save')` auto-extracts 2-letter uppercase monogram initials (e.g., "Elena Rostova" → "ER").

3.2 REST API Endpoints

Method	Endpoint	Auth Required	Description
`POST`	`/api/auth/signup`	No	Registers user, hashes password, returns JWT token & user profile.
`POST`	`/api/auth/login`	No	Verifies identifier (email or username) + password, returns JWT token.
`GET`	`/api/auth/me`	Yes (Bearer JWT)	Returns profile data for the authenticated session.
`PUT`	`/api/auth/profile`	Yes (Bearer JWT)	Updates names, username, and phone; re-issues refreshed token.
`GET`	`/api/cards`	Yes (Bearer JWT)	Lists all cards belonging to the logged-in `req.user.id`.
`GET`	`/api/cards/:id`	Yes (Bearer JWT)	Retrieves a specific card belonging to the user.
`POST`	`/api/cards`	Yes (Bearer JWT)	Creates a new card linked to `req.user.id`.
`PUT`	`/api/cards/:id`	Yes (Bearer JWT)	Updates card details owned by `req.user.id`.
`DELETE`	`/api/cards/:id`	Yes (Bearer JWT)	Deletes card owned by `req.user.id`.

4. All Your Commits (Author: `riddhii24`) Grouped by Feature

> **Total Contribution Summary: 25 commits across July – October 2026**
>
> Author Identity: `riddhii24 <riddhigandhi72@gmail.com>`

Group 1: Initial Foundation & Workspace Architecture (July 2026)

- `4f67bec` (2026-07-22) — *home page*
  Files: `package.json`, `README.md`, `main/index.html`, `main/dashboard.css`, `main/app.js`.
  What & Why: Initialized npm workspace, dependencies, and created the foundational layout and prototype styles.
- `00b14d3` (2026-07-22) — *Home Page*
  Files: `index.html`, `dashboard.css`, `app.js`, `.vscode/settings.json`.
  What & Why: Added root landing prototype and editor configuration.
- `badc556` (2026-07-23) — *Project Structure*
  Files: `README.md`.
  What & Why: Formally defined directory conventions in project documentation.
- `25145d2` (2026-07-23) — *Main*
  Files: Deleted redundant root files (`app.js`, `dashboard.css`, `index.html`).
  What & Why: Cleaned up root clutter to prepare for clean Client/Server modularization.
- `c4e3ed5` (2026-07-23) — *(Dependency update)*
  Files: `package.json`, `package-lock.json`.
  What & Why: Synchronized npm dependencies.
- `68de2d1` (2026-07-28) — *(Folder restructure)*
  Files: Moved files into `Client/Pages/login.html`, `Client/dashboard.css`, `Client/index.html`, updated `README.md`.
  What & Why: Established strict separation between frontend client pages and backend server directories.

Group 2: Express Server Setup & Authentication Foundations (August 2026)

- `f9f7ace` (2026-08-06) — *feat: add Express server, light/dark mode toggle, login→log in, email/username field, fix nav logo, remove PRO badge, link dashboard.css*
  Files: `Server/server.js`, `Client/Pages/dashboard.html`, `Client/Pages/login.html`, `Client/Pages/login.js`, `dashboard.css`, `.gitignore`.
  What & Why: Converted the project into an Express.js web server with static routing, introduced the Light/Dark theme toggle script, and updated login credentials to accept username or email.
- `b66055b` (2026-08-06) — *update*
  Files: Created `Client/app.js`, added `Server/login.js`.
  What & Why: Centralized client-side AngularJS logic and backend login handling.

Group 3: Dashboard UI, Liquid Dock Navigation & Styling (August 2026)

- `f98bc3d` (2026-08-08) — *Update dashboard*: Linked updated stylesheets to `dashboard.html`.
- `7c2130f` (2026-08-08) — *dashboard update*: Added +260 lines to `Client/app.js` and +168 lines to `Client/style.css`.
- `2b97669` (2026-08-08) — *dashboard update*: Enhanced signup form inputs, refined `app.js`, and recorded reference dock video.

- `51f80a4` (2026-08-08) — *dashboard update*: Consolidated CSS files into `Client/Style/` (`dashboard.css` , `signup.css` , `style.css`).
- `91d7d95` (2026-08-10) — *(Styles & controllers)*: Updated controllers in `app.js` and server routing paths.
- `0c61667` (2026-08-12) — *(Signup tweaks)*: Polished form input validation attributes in `signup.html` .
- `7949a91` (2026-08-12) — *(Server route adjustments)*: Configured HTTP cache prevention headers in `Server/server.js` .
- `927edb0` (2026-08-12) — *removed navigation bar*: Cleaned up legacy top navbar in favor of the floating Liquid Dock.
- `5c03cdf` (2026-08-12) — *change in card animation*: Smoothed 3D perspective tilt calculations and transitions on the hero card.
- `78dfa9f` & `966fc45` (2026-08-08) — *Merge branch 'main'*: Resolved team repository branches cleanly.

Group 4: Guest Preview & Showcase System (September 2026)

- `425b7d1` (2026-09-26) — *Guest auhtentication*: Implemented guest checks across `create-card.html` and `dashboard.html` .
- `e876a11` (2026-09-26) — *sample card modified*: Refined showcase cards (Elena Rostova, Julian Vance) for guest first-load experience.
- `35d5cd3` (2026-09-27) — *Guest preview changed*: Reorganized auth files to server and polished guest preview interactions.

Group 5: Full Guest Auth Architecture & Dynamic Fixes (Major Milestone, October 2026)

- `50ededc` (2026-10-25) — *guest auth*
  Files Touched: Created `Client/services/auth.service.js` , `Client/services/auth-modal.service.js` , `Client/components/auth-gate.component.js` , `Client/routes.config.js` , `Client/Style/auth-modal.css` , modified `Client/app.js` (+309 lines), integrated across all 7 page views.
  What & Why: Built the complete guest authentication and route gatekeeping system. Intercepts unauthorized navigation to member pages, saves target routes, broadcasts events to open the `<auth-gate>` modal, and handles post-login return routing.
- `061be27` (2026-10-26) — *dynamic issue fix*
  Files Touched: `create-card.html` , `dashboard.html` , `saved-cards.html` , `scan.html` , `templates.html` , `studio.css` , `style.css` .
  What & Why: Fixed dynamic card data rendering, resolved template preview synchronization, and corrected CSS layout bugs across the studio and scanner.
- `c6984c6` (2026-10-26) — *dashboard and saved cards*
  Files Touched: `Client/Pages/dashboard.html` (+340 lines), `Client/Pages/saved-cards.html` (+42 lines), reference showcase videos.
  What & Why: Engineered the animated expandable Search Pill with ripple effect, completed saved cards catalog views, and recorded video demos.

5. Summary of Your Total Contribution

As **Riddhi Gandhi** (`riddhii24`), your work spans full-stack engineering with core leadership in:

1. **Authentication & Route Security Architecture:** Designed and implemented the complete **Guest Mode & Protected Route System** (`auth.service.js` , `routes.config.js` , `auth-gate.component.js` , `auth-modal.service.js`). Guests can explore features without friction, while member routes are intercepted by the `$transitions.onStart` gatekeeper modal with automatic redirect handling.
2. **Backend Express Server:** Built the initial Express.js backend (`Server/server.js`), static file distribution, cache-control headers, and page route handlers.
3. **Tactile UI & Interaction Engineering:** Built the **Expandable Animated Search Pill** with custom ripple animation on the dashboard, and implemented the **Light/Dark Mode Theme Toggle** with `localStorage` persistence.
4. **Catalog Logic & Data Synchronization:** Engineered client-side multi-parameter filtering (by text, company, template, category) in `SavedCardsController` and fixed cross-page dynamic template synchronization.

6. 20 Likely College Viva Questions and Answers

Q1. In commit 50ededc, you introduced "guest auth". Why did you build this architecture instead of forcing login immediately?

Answer: Forcing registration upfront creates severe onboarding friction. Our guest architecture lets visitors immediately interact with 3D cards, test templates, and explore features. The custom `authGate` modal is only triggered when users attempt restricted member operations (saving cards to the cloud, accessing the personal vault, or running the camera scanner).

Q2. How does your route protection interceptor work in routes.config.js?

Answer: It hooks into AngularJS ui-router's `$transitions.onStart` event. For every state navigation, it checks `toState.data.requiresAuth`. If `true` and `AuthService.isLoggedIn()` returns `false`, it saves the attempted route via `AuthService.setRedirectTarget()`, emits an `'authGate:open'` event via `AuthModalService` to display the modal, and cancels navigation with `return false`.

Q3. In commit f9f7ace, you added the Light/Dark mode switcher. How does it work under the hood?

Answer: It toggles the `data-theme="dark"` attribute on the root `document.documentElement` and persists the choice to `localStorage.getItem('inoviq_theme')`. In `style.css`, tactile CSS variables (e.g. `--bg-page`, `--text-primary`, `--paper-card`) update dynamically without requiring a page reload.

Q4. In commit 927edb0, you updated the navigation layout. How did you improve it?

Answer: We removed bulky legacy navbar elements in favor of a clean, minimalist brand header featuring the Inoviq logo, primary route links, the animated expandable Search Pill (with circular ripple animation), and a classic footer. This eliminated viewport clutter and ensured mobile screens and card canvas editing have maximum visible work area.

Q5. What was the "dynamic issue fix" in commit 061be27?

Answer: When switching between the Templates showcase, the 3D Card Studio, and Saved Cards, template parameters were falling out of sync with AngularJS `$scope` bindings. The fix ensured that template style objects, color codes, and cardholder metadata correctly re-hydrated form inputs and canvas previews dynamically across all pages.

Q6. What does the expandable search pill in commit c6984c6 do?

Answer: It is an animated search element on the dashboard (`dash-search`). In its default state, it is a compact 44px circular button. When clicked or focused, it fires a CSS keyframe circular ripple animation and smoothly expands to a 320px input pill using cubic-bezier easing. Pressing `Escape` or clicking outside clears the search and collapses it.

Q7. Why does the project use both AngularJS and React 18?

Answer: It is a hybrid architecture. **AngularJS 1.8.3** manages single-page routing, controllers, and data-binding across the main application shell. **React 18** is mounted specifically in `create-card.html` for the **3D Card Studio**, providing a high-performance reactive canvas with zero layout lag during real-time 3D cursor tilt calculations and drag-and-resize transformations.

Q8. How is the 3D card tilt and specular glare effect implemented?

Answer: The canvas container has CSS `perspective: 1200px`, and the card has `transform-style: preserve-3d`. On `mousemove`, JavaScript computes cursor distance from card center and applies CSS `transform: rotateX(...) rotateY(...)`. A pseudo-element or overlay with a CSS `radial-gradient` shifts position according to mouse coordinates to create the realistic glare highlight.

Q9. What is the difference between "Edit Mode" and "3D Preview Mode" in the Studio?

Answer: In **Edit Mode**, 3D tilt is disabled (`transform: none`) so the card remains flat and stationary, allowing accurate typing, logo uploads, and drag adjustments. In **3D Preview Mode**, cursor tilt physics and specular glare are active so users can evaluate the tactile appearance of their card in 3D space.

Q10. How is the contact QR code generated, and what data format does it use?

Answer: It uses the client-side `qrcodejs` library. It formats contact credentials into standard **vCard** or **MECARD** strings (`BEGIN:VCARD ... FN: ... ORG: ... TEL: ... EMAIL: ... END:VCARD`). When scanned by any smartphone camera, the operating system natively recognizes the payload and prompts to "Add to Contacts".

Q11. How does the Camera Scanner in scan.html work?

Answer: It uses `Html5Qrcode` to access the device's rear environment camera via `navigator.mediaDevices.getUserMedia`. When a QR code is detected, regular expressions extract name, organization, phone, and email fields, automatically populating the form for the user to review and save.

Q12. How are cards exported to PNG images?

Answer: The studio uses `html2canvas`. It rasterizes the card's DOM tree into an HTML5 `<canvas>` element at 3x scale for crisp resolution, converts the canvas into a PNG data URL (`canvas.toDataURL('image/png')`), and triggers a temporary `<a download>` link click.

Q13. How are user passwords secured in MongoDB?

Answer: In `Server/models/User.js`, a Mongoose `pre('save')` hook intercepts the password before saving. If modified, it generates a salt with `bcrypt.genSalt(12)` and hashes the password with `bcrypt.hash()`. Cleartext passwords are never stored in the database.

Q14. How does the JWT authentication flow work?

Answer: When a user logs in via `POST /api/auth/login`, the server verifies their credentials with `bcrypt.compare()`. If valid, it signs a JSON Web Token containing the user ID and email using `jwt.sign()` with an expiration of 7 days. The client stores this token in `localStorage` and includes it in an `Authorization: Bearer <token>` header for protected API calls.

Q15. How does authMiddleware protect API endpoints?

Answer: In `Server/middleware/auth.js`, it reads `req.headers.authorization`. If the header is missing or does not start with `Bearer`, it rejects the request with HTTP 401 Unauthorized. Otherwise, it calls `jwt.verify(token, process.env.JWT_SECRET)` and attaches the decoded user payload to `req.user` before calling `next()`.

Q16. How does the server handle MongoDB Atlas cloud connection failures?

Answer: In `Server/config/db.js`, the connection function sets a 5-second timeout (`serverSelectionTimeoutMS: 5000`). If Atlas is unreachable (e.g. IP whitelist restrictions or offline), a `catch` block automatically attempts a fallback connection to local MongoDB at `mongodb://127.0.0.1:27017/inoviq` so the development server stays alive.

Q17. How is data isolation ensured between different users' cards?

Answer: In `Server/routes/cards.js`, every query filters cards by `userId: req.user.id`. For example: `Card.find({ userId: req.user.id })` or `Card.findOneAndDelete({ _id: req.params.id, userId: req.user.id })`. This ensures a user can never view, edit, or delete another user's cards.

Q18. How are card initials generated in the database model?

Answer: In `Server/models/Card.js`, a `pre('save')` hook splits `fullName` by whitespace. If two or more words exist, it takes the first letter of each (e.g., "Elena Rostova" → "ER"). If single-word, it takes the first two letters. If blank, it defaults to `"IN"`.

Q19. How does the frontend handle offline or unauthenticated card listing in SavedCardsController?

Answer: If the user is a guest or the backend is unreachable, `SavedCardsController` initializes `catalog.cards` with an array of built-in sample cards (`defaultShowcase`). This ensures the UI remains functional, interactive, and demonstrable during reviews or offline evaluations.

Q20. What HTTP status codes are used in your REST API?

Answer: `200 OK`: Successful fetch or update. `201 Created`: Successful resource creation (signup, new card). `400 Bad Request`: Missing required fields or password too short. `401 Unauthorized`: Missing or invalid JWT token. `404 Not Found`: Target card or user does not exist. `409 Conflict`: Duplicate email or username during registration. `500 Internal Server Error`: Uncaught database or server error.